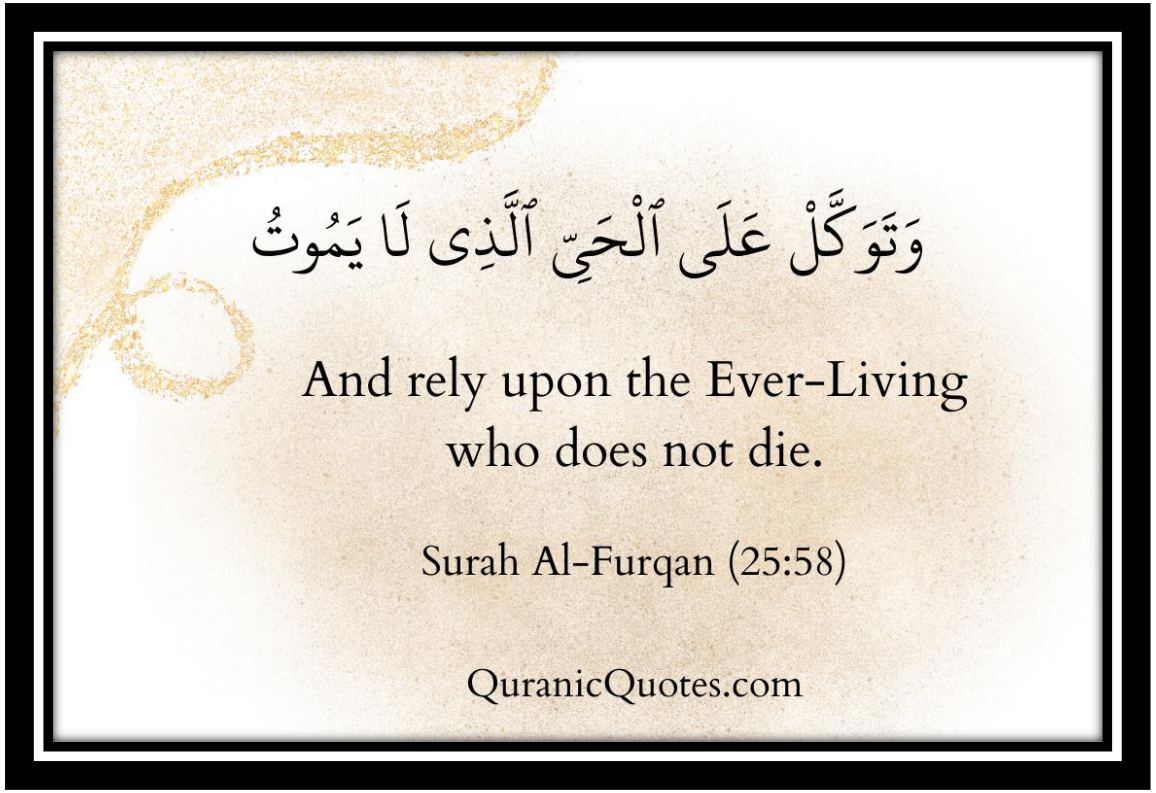
Parallel & Distributed Computing

GPU ARCHITECTURES & CUDA/OPENCL BASICS

Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran



Lecture Outline

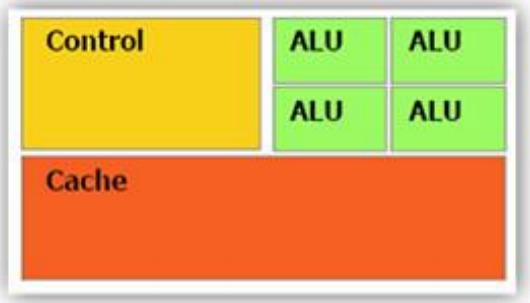
- Why GPU Architectures Matter
- CPU vs GPU Design Goals
- GPU as a Heterogeneous Accelerator
- Internal GPU Architecture
- Memory Bandwidth vs Latency
- CUDA Programming Overview
- OpenCL Programming Model
- CUDA vs OpenCL (Technical View)

Why GPU Architectures Matter

- Moore’s Law slowdown limits CPU performance gains
- Frequency scaling no longer viable due to power/thermal limits
- Performance growth now depends on **parallelism**, not clock speed
- GPUs provide massive thread-level parallelism
- Ideal for workloads with:
  - High arithmetic intensity (Matrix Multiplication)
  - Regular memory access patterns
  - Minimal control divergence (Image Brightness Adjustment)
- GPUs dominate modern:
  - AI accelerators
  - Scientific computing
  - Real-time rendering
  - Data analytics

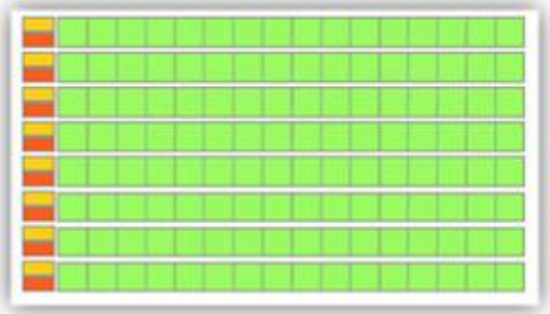
CPU vs GPU Design Goals

CPU



* Low compute density
* Complex control logic
* Large caches (L1$/L2$, etc.)
* Optimized for serial operations
  - Fewer execution units (ALUs)
  - Higher clock speeds
* Shallow pipelines (<30 stages)
* Low Latency Tolerance
* Newer CPUs have more parallelism

GPU



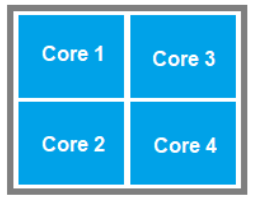
* High compute density
* High Computations per Memory Access
* Built for parallel operations
  - Many parallel execution units (ALUs)
  - Graphics is the best known case of parallelism
* Deep pipelines (hundreds of stages)
* High Throughput
* High Latency Tolerance
* Newer GPUs:
  - Better flow control logic (becoming more CPU-like)
  - Scatter/Gather Memory Access
  - Don't have one-way pipelines anymore

CUDA's Core Idea of Parallelism

- The fundamental idea of CUDA is simple:
  - Instead of using a loop to process data elements sequentially,
  - CUDA assigns one GPU thread to each data element.
- Thousands of GPU threads execute the same kernel function simultaneously, each working on a different portion of the data.
- This model converts a sequential loop into a massively parallel operation without changing the core logic of the computation.

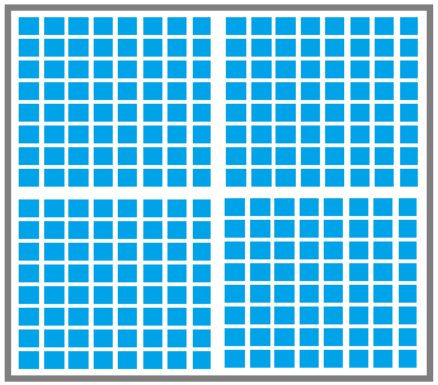
Architectural Resource Allocation

- CPU silicon area mostly devoted to:
  - Control units
  - Cache memory
  - Branch predictors
- GPU silicon area mostly devoted to:
  - ALUs
  - Floating-point units
  - Parallel execution cores
- Result:
  - CPUs execute fewer threads very fast
  - GPUs execute many threads moderately fast



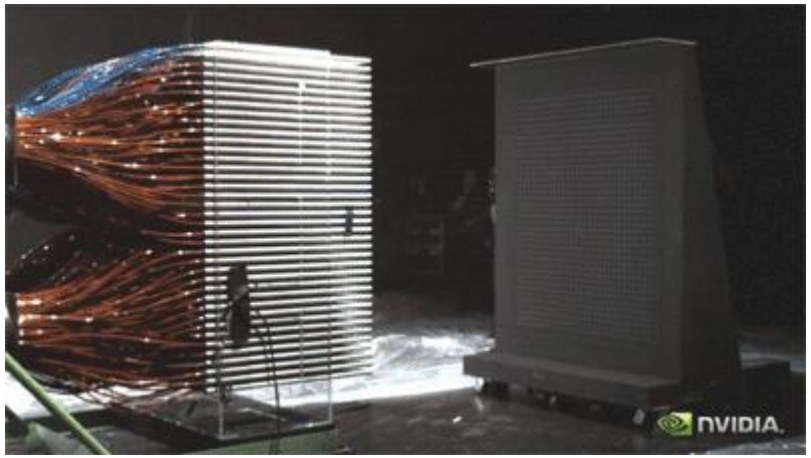
(Fewer strong cores)

CPU



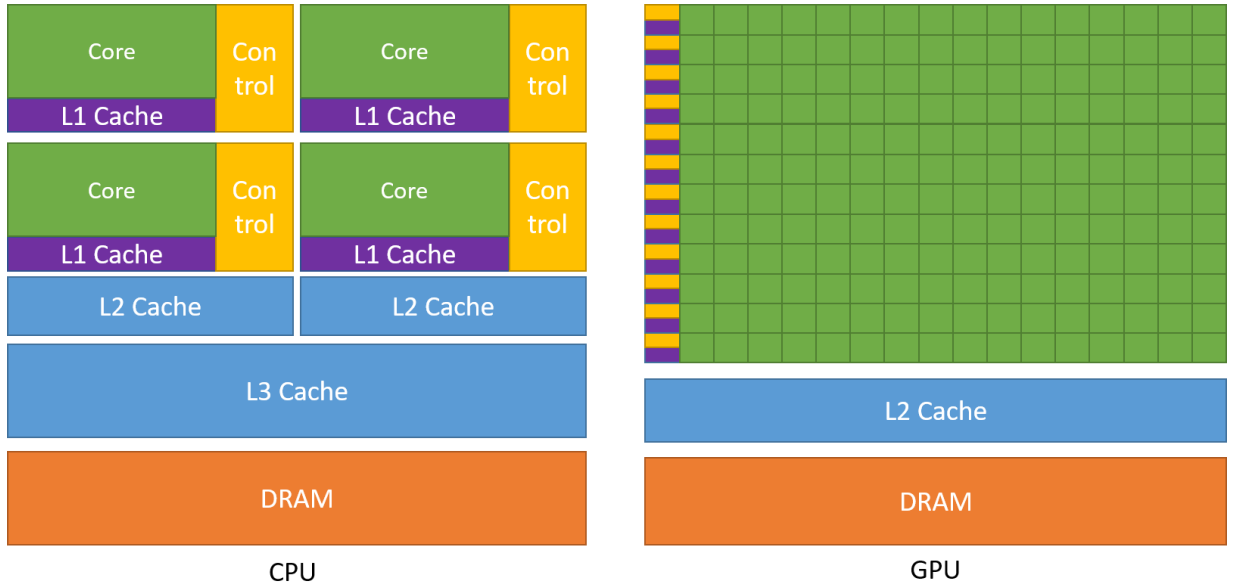
(Thousands weaker cores)

GPU



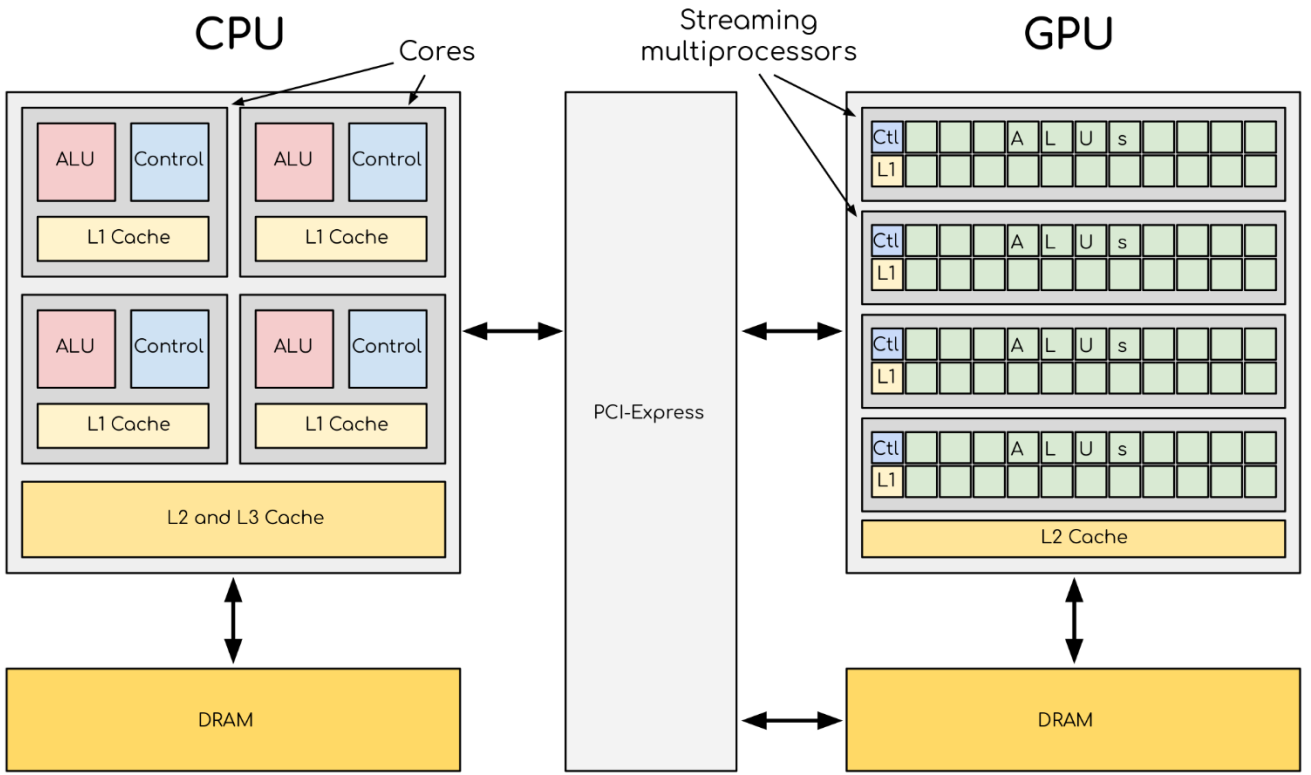
GPU as a Heterogeneous Accelerator

- GPU does not replace CPU
- CPU:
  - Executes sequential code
  - Manages OS and I/O
  - Launches GPU kernels
- GPU:
  - Executes compute-intensive kernels
  - Handles parallel loops and vector operations
- Communication via:
  - PCIe
  - Unified Virtual Addressing
- Programming model is **heterogeneous**



Internal GPU Architecture

- GPU consists of multiple Streaming Multiprocessors (SMs)
- Each SM contains:
  - Many execution cores
  - Register files
  - Shared memory
  - Warp schedulers
- SMs execute blocks independently
- Scalability achieved by adding more SMs

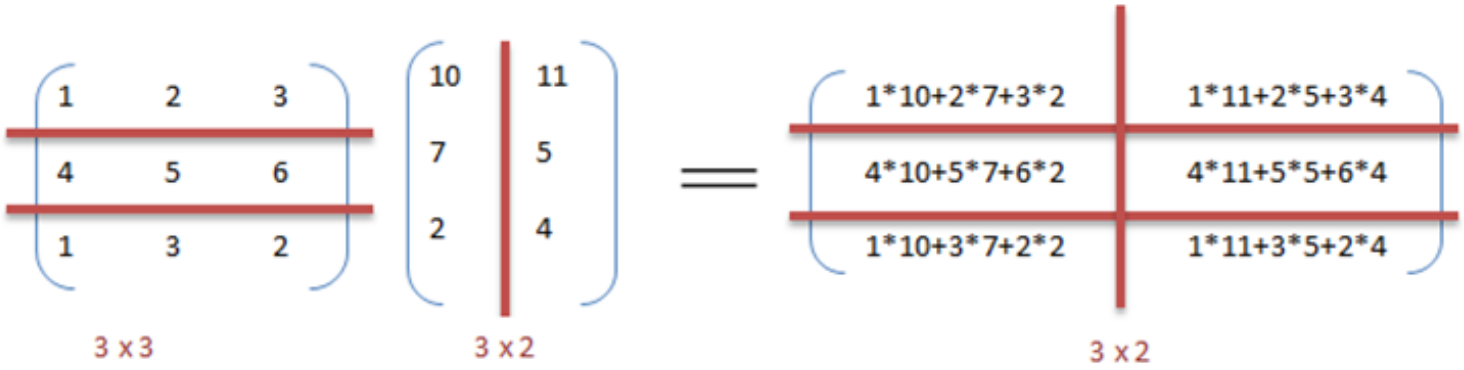


Memory Bandwidth vs Latency

- GPU global memory latency is very high
- GPUs compensate by:
  - Executing thousands of threads
  - Overlapping memory access with computation
- Bandwidth is more important than latency
- Performance depends on:
  - Memory coalescing
  - Data reuse

Parallelism on Accelerators

- GPUs exploit **data parallelism**
- Same instruction applied to different data
- Thousands of threads execute kernel code
- Threads are lightweight compared to CPU threads
- Suitable for:
  - Vector operations
  - Matrix computations
  - Image processing



SIMD vs SIMT Execution

- **SIMD**
- Single instruction, multiple data lanes
- Used in CPU vector units
- Fixed-width execution
- **SIMT**
- Single instruction, multiple threads
- Each thread has its own registers
- Threads grouped into warps
- More flexible than SIMD

Warp Execution and Divergence

- Warp executes one instruction at a time
- All threads in warp must follow same control path
- Branch divergence:
  - Occurs when threads take different branches
  - Causes serialization
- Reduces parallel efficiency
- Good GPU code minimizes divergence

Examaple: Suppose you have an if statement:

```
if (thread_id % 2 == 0)
    do_task_A();
else
    do_task_B();
```

Half the threads want task_A, half want task_B.

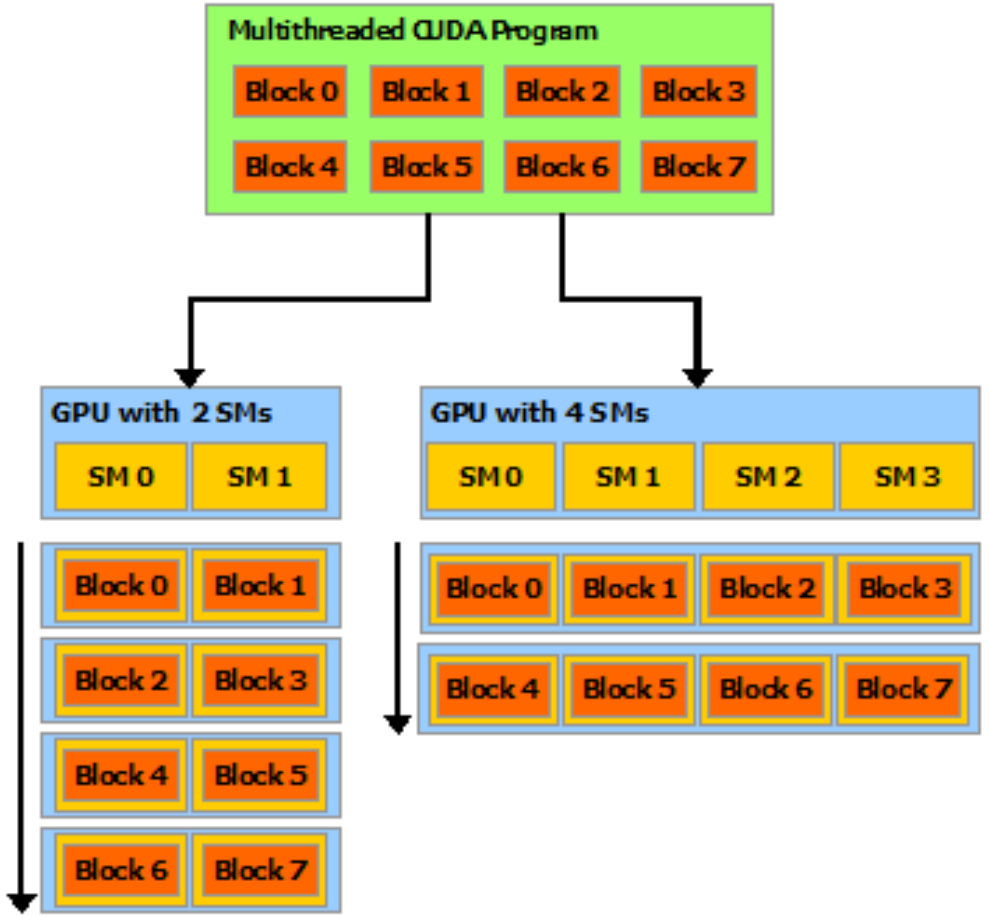
The GPU cannot do both at once for the warp.

It first runs task_A for threads that need it, then runs task_B for the others.

This serialization wastes time → slower performance.

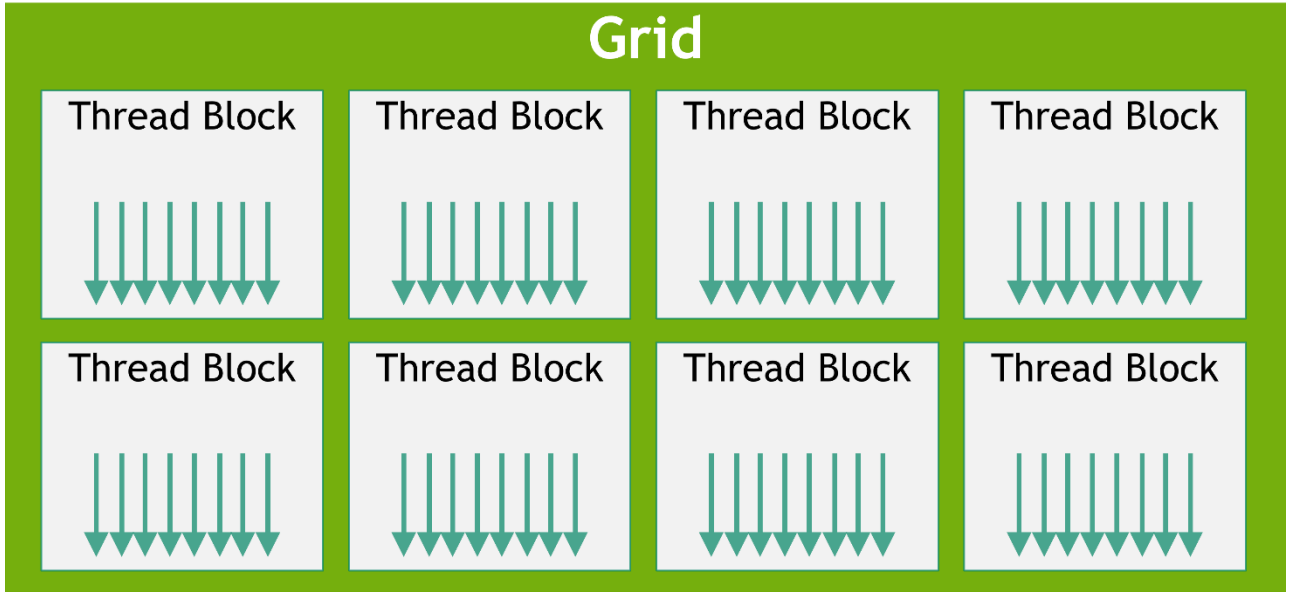
CUDA Programming Overview

- CUDA = Compute Unified Device Architecture
- NVIDIA proprietary platform
- Extends C/C++ with GPU constructs
- Programmer explicitly manages:
  - Parallelism
  - Memory movement
- Kernel functions run on GPU



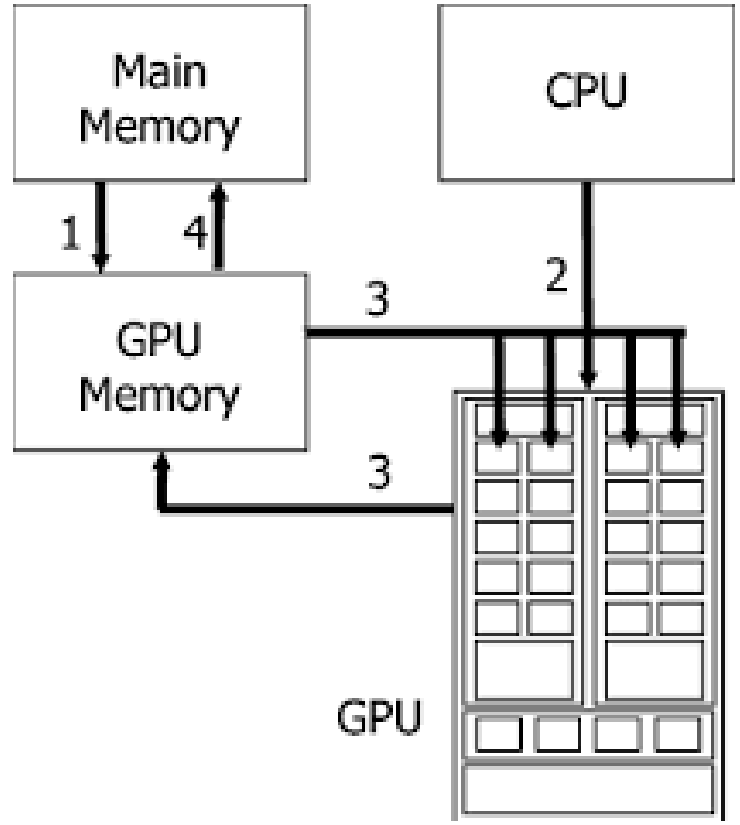
CUDA Thread Hierarchy

- Grid:
  - Entire kernel launch
- Blocks:
  - Independent execution units
  - Scheduled to SMs
- Threads:
  - Execute kernel instructions
- Hierarchy enables scalable parallelism
- Same kernel works on small or large GPUs



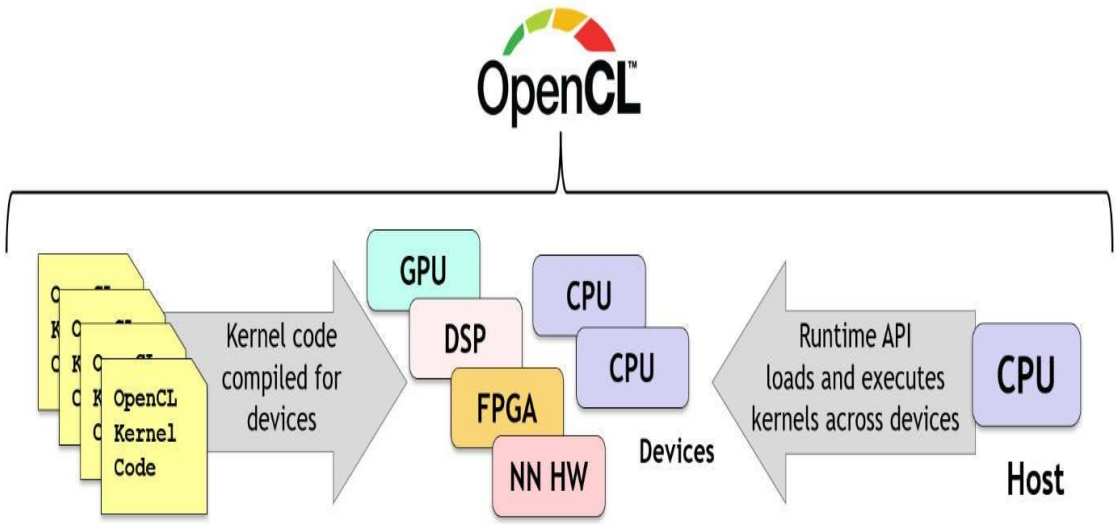
CUDA Kernel Execution Flow

- CPU allocates memory
- Data copied from host to device
- Kernel launched with grid and block configuration
- GPU schedules blocks onto SMs
- Threads execute kernel
- Results copied back to host



OpenCL Programming Model

- Open standard for heterogeneous computing
- Supports CPUs, GPUs, FPGAs
- Platform-independent
- Explicit management of:
  - Devices
  - Contexts
  - Memory buffers
- More verbose than CUDA

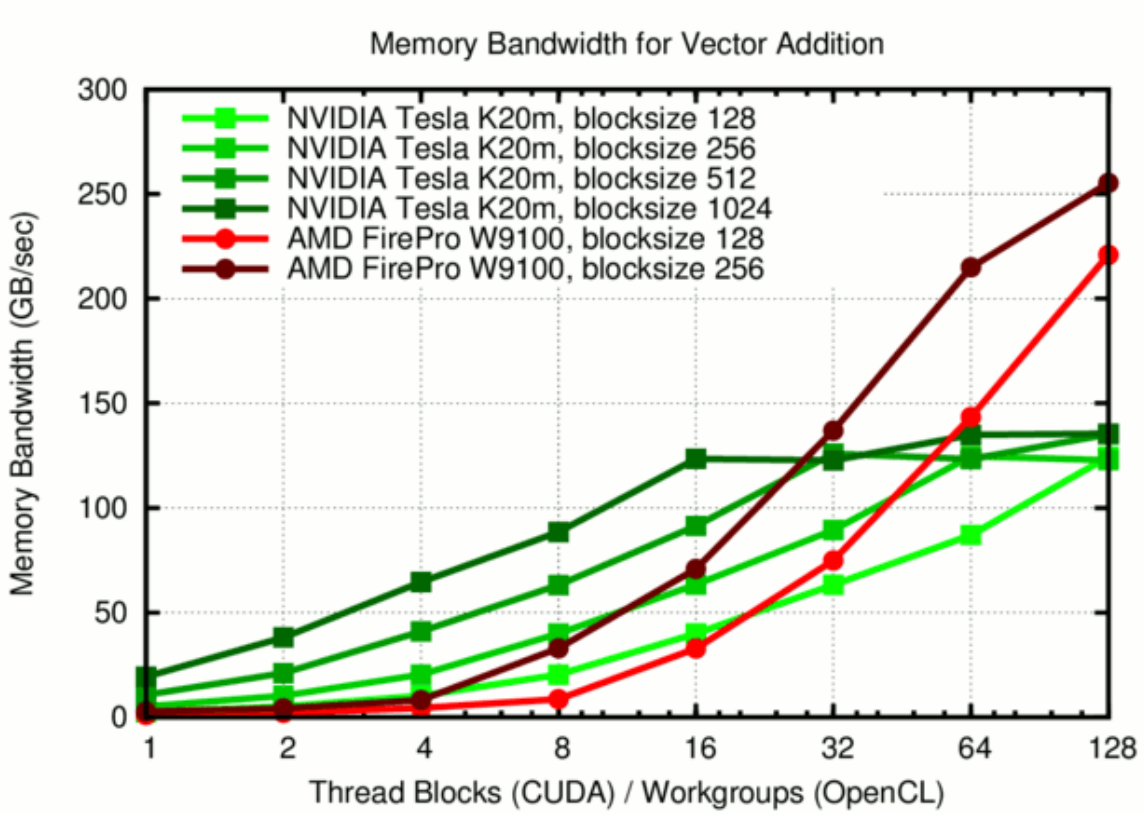


OpenCL Execution Structure

- Host program controls execution
- Kernels written in OpenCL C
- Work-items ≈ threads
- Work-groups ≈ thread blocks
- NDRange defines global and local sizes

CUDA vs OpenCL (Technical View)

- CUDA:
  - Higher-level abstractions
  - Mature debugging tools
  - Vendor-locked to NVIDIA
- OpenCL:
  - Portability across devices
  - Lower-level control
  - Increased programming complexity



Summary

- GPUs are throughput-oriented accelerators
- Architecture designed for massive parallelism
- SIMT model enables flexible parallel execution
- CUDA and OpenCL are core accelerator models
- Performance depends on:
  - Parallelism
  - Memory efficiency
  - Architecture-aware design

Thanks